

Demo - Spring Boot + Angular

Projekt demonstracyjny z backendem w Spring Boot i frontendem w Angularze. Backend udostępnia API do logowania, rejestracji i zarządzania produktami, a frontend pozwala zalogować się i dla użytkownika z rolą **ADMIN** - dodać nowy produkt.

Struktura projektu

- **src/main/java** - backend Spring Boot
- **src/main/resources/application.properties** - konfiguracja backendu
- **frontend/** - aplikacja Angular

Jak działa backend

Backend jest zbudowany warstwowo:

- **controller** przyjmuje requesty HTTP i zwraca odpowiedzi JSON
- **service** zawiera logikę biznesową
- **repository** komunikuje się z bazą danych
- **dto** opisuje dane wejściowe i wyjściowe dla API
- **security** odpowiada za JWT, uwierzytelnianie i autoryzację
- **model** opisuje encje JPA zapisywane w bazie

Przebieg requestu

1. Frontend wysyła request do endpointu REST.
2. Spring Security sprawdza, czy endpoint wymaga logowania.
3. Jeśli endpoint jest chroniony, backend oczekuje nagłówka:

```
Authorization: Bearer <token>
```

4. **JwtAuthenticationFilter** odczytuje token, weryfikuje podpis i ustawia użytkownika w kontekście Security.
5. Kontroler przekazuje dane do serwisu.
6. Serwis waliduje dane biznesowe i zapisuje lub pobiera rekordy z bazy.
7. Odpowiedź wraca do frontend jako JSON.

Najważniejsze moduły backendu

Autoryzacja i logowanie

- **AuthController.java** obsługuje **/api/auth/register** i **/api/auth/login**
- **JwtUtil.java** generuje i waliduje token JWT
- **JwtAuthenticationFilter.java** sprawdza nagłówek **Authorization**
- **CustomUserDetailsService.java** pobiera użytkownika z bazy na podstawie loginu
- **SecurityConfig.java** definiuje zasady dostępu do endpointów

Rejestracja

Endpoint `/api/auth/register`:

- przyjmuje `username`, `email`, `password`, `role`
- sprawdza, czy login i email nie istnieją już w bazie
- hasło jest hashowane przez `PasswordEncoder`
- rola zapisywana jest jako `USER` albo `ADMIN`
- po zapisie backend publikuje event o nowym użytkowniku

Logowanie

Endpoint `/api/auth/login`:

- przyjmuje `username` i `password`
- `AuthenticationManager` sprawdza dane logowania
- jeśli są poprawne, backend generuje token JWT
- token zawiera:
 - `subject` = login użytkownika
 - `claim role` = rola użytkownika
- frontend zapisuje token w `localStorage`

Produkty

- `ProductController.java` obsługuje `/api/products`
- `ProductService.java` waliduje i zapisuje produkty
- `ProductMapper.java` mapuje DTO na encje i odwrotnie
- `ProductRequest.java` opisuje dane do tworzenia produktu
- `ProductResponse.java` opisuje dane zwracane do frontend

Pobieranie produktów

- `GET /api/products` zwraca listę wszystkich produktów
- endpoint jest chroniony przez Spring Security
- frontend musi wysłać poprawny token JWT

Dodawanie produktu

- `POST /api/products` wymaga roli `ADMIN`
- kontroler ma adnotację `@PreAuthorize("hasRole('ADMIN')")`
- request musi zawierać:
 - `name`
 - `price`
 - `description`
- serwis sprawdza:
 - czy nazwa nie jest pusta
 - czy cena nie jest ujemna
 - czy obiekt nie jest `null`

Baza danych

Projekt używa H2 w pamięci.

- baza startuje razem z aplikacją
- dane nie są trwale zapisywane po restarcie
- Hibernate ma ustawione `ddl-auto=update`, więc tworzy i aktualizuje tabele automatycznie

Wymagania

- Java 21 lub nowsza
- Maven 3.9+ albo Maven Wrapper (`mvnw`)
- Node.js 18+
- npm
- przeglądarka internetowa

Uruchomienie backendu

Backend działa domyślnie na porcie `8080`.

Windows

```
mvnw.cmd spring-boot:run
```

Linux / macOS

```
./mvnw spring-boot:run
```

Po starcie aplikacja wystawia API pod adresem:

- `http://localhost:8080/api/auth`
- `http://localhost:8080/api/products`

H2 Console

Projekt używa bazy H2 w pamięci. Konsola H2 jest włączona i dostępna pod:

- `http://localhost:8080/h2-console`

W konfiguracji użyj:

- JDBC URL: `jdbc:h2:mem:testdb`
- User Name: domyślnie `sa`
- Password: puste

Uruchomienie frontendu

Frontend znajduje sie w katalogu `frontend/` i dziala domyslnie na porcie `4200`.

Instalacja zaleznosci

```
cd frontend
npm install
```

Start aplikacji

```
npm start
```

albo:

```
ng serve
```

Frontend korzysta z proxy Angulara i wysyla requesty na wzgledne adresy `/api/...`, ktore sa przekierowywane do backendu.

Konfiguracja backendu

Najwazniejsze ustawienia znajduja sie w:

- `src/main/resources/application.properties`

Aktualne parametry

- `spring.datasource.url=jdbc:h2:mem:testdb`
- `spring.datasource.driver-class-name=org.h2.Driver`
- `spring.h2.console.enabled=true`
- `spring.jpa.hibernate.ddl-auto=update`
- `spring.jpa.show-sql=true`
- `jwt.secret=...`
- `jwt.expiration-ms=86400000`

Co to oznacza

- baza danych jest in-memory, wiec po restarcie dane znikaja
- JWT wygasa po 24 godzinach
- backend uzywa wlasnego sekretu do podpisywania tokenow

Logowanie i autoryzacja

- `/api/auth/register` - rejestracja uzytkownika
- `/api/auth/login` - logowanie i zwrot tokenu JWT
- `/api/products` - pobieranie listy produktow

- **POST /api/products** - dodawanie produktu, wymaga roli **ADMIN**

Frontend zapisuje token JWT w **localStorage** i dołącza go do requestow do API jako naglowek:

```
Authorization: Bearer <token>
```

Uwagi

- Zwykly uzytkownik moze zobaczyc liste produktow.
- Dodawanie produktu jest dostepne tylko dla konta z rola **ADMIN**.
- Jesli w przegladarce pojawia sie **401 Unauthorized**, najczesciej oznacza to brak tokenu, stary token albo brak wymaganej roli.

Zgodnosc z kryteriami oceny

Ocena 3.0

- dziala backend Spring Boot
- projekt laczy sie z baza danych H2
- istnieja operacje na encjach przez REST API
- demo video

Jak dokladnie:

- **Spring Boot** startuje aplikacje i udostepnia REST API.
- Polaczenie z baza jest w **src/main/resources/application.properties** przez **jdbc:h2:mem:testdb**.
- Encja glowna to **Product** z JPA, a dane sa zapisywane przez **ProductRepository**.

Ocena 4.0

- zachowana jest struktura warstwowa **controller / service / repository**
- dane wejscia i wyjscia sa opisane przez DTO
- pola sa walidowane przez adnotacje **jakarta.validation**
- obsluga bledow zwraca czytelne odpowiedzi HTTP
- zabezpieczenie dziala na bazie JWT

Jak dokladnie:

- **Controller / Service / Repository**
 - **AuthController** przyjmuje requesty **/api/auth/register** i **/api/auth/login**
 - **ProductController** obsluguje **/api/products**
 - **ProductService** zawiera logike biznesowa i walidacje
 - **ProductRepository** i **UserRepository** komunikuja sie z baza
- DTO
 - **LoginRequest**, **RegisterRequest**, **ProductRequest**, **ProductResponse** oddzielaja format API od encji JPA
- Walidacja

- `@NotBlank`, `@Email`, `@Size`, `@Pattern`, `@Min` pilnują poprawności danych
- błędne dane zwracają `400 Bad Request`
- Obsługa błędów
 - konflikty przy rejestracji zwracają `409`
 - brak autoryzacji zwraca `401`
 - backend ma zdefiniowane odpowiedzi w `SecurityConfig` i kontrolerach
- Security
 - login tworzy JWT w `JwtUtil`
 - `JwtAuthenticationFilter` sprawdza nagłówek `Authorization`
 - `SecurityConfig` chroni wszystko poza `/api/auth/**`
 - `@PreAuthorize("hasRole('ADMIN')")` zabezpiecza dodawanie produktu

Ocena 5.0

- testy jednostkowe serwisów
- event po rejestracji użytkownika
- frontend Angular konsumujący API
- kod jest podzielony na czytelne warstwy i klasy odpowiedzialności

Jak dokładnie:

- Testy jednostkowe
 - testy dla `ProductService` sprawdzają zapis, wyszukiwanie i walidacje
 - testy używają Mockito do mockowania repozytorium
 - znajdują się w `src/test/java/com/example/demo/...`
- Events
 - po rejestracji w `AuthController` jest publikowany `UserRegisteredEvent`
 - `UserRegisteredEventListener` reaguje na ten event i loguje dane nowego użytkownika
- Frontend Angular
 - frontend wysyła requesty do backendowego API przez `AuthService` i `ProductService`
 - JWT jest zapisywany w `localStorage`
 - token jest dołączany do requestów jako `Authorization: Bearer <token>`
- Czysty podział odpowiedzialności
 - encje są w `model`
 - dane API są w `dto`
 - mapowanie jest w `mapper`
 - logika w `service`
 - endpointy w `controller`
 - auth w `security`